

Guía de uso para la simulación Create3

El actual documento explica como ejecutar una simulación en Ubuntu y ROS2 Humble con el robot Create3 usando Reinforcement Learning (RL) y SLAM.

El código necesario está dividido en dos repositorios de github:

- **create3_sim:** *Este repositorio contiene el código necesario para ejecutar una simulación del robot Create3 usando Gazebo, con un sensor LIDAR y un módulo de SLAM.*
- **turtlebot3_drlnav:** *Este repositorio contiene el código necesario para ejecutar el módulo de RL.*

Instalación:

Lo primero que tendremos que hacer será instalar el código fuente en nuestro equipo. El código ha sido pensado para trabajar con ROS2 Humble y ha sido probado en un equipo con Ubuntu 22.04 Jammy. Nota: En mi caso tengo instalados los repositorios en workspace diferentes, pero no debería de influir.

1. Instalar ROS2 humble (<https://docs.ros.org/en/humble/Installation.html>)
2. Instalar create3_sim (rama drl-nav). Simplemente seguir las instrucciones en el readme del repo en las secciones Prerequisites y Build. Usar Gazebo classic. Adjunto un requirements.txt generado con pipreqs en el root del repo con las versiones de las dependencias con las que se ha probado el código por si fuese necesario.
3. Instalar turtlebot3_drlnav (rama drl-env-vel). Adjunto un requirements.txt. Descargar el código fuente y ejecutar los siguientes comandos antes de compilar el código usando colcon build en la base del repo:

```
pip3 install pandas
pip3 install pyqtgraph==0.12.4
pip3 install PyQt5==5.14.1
pip3 install torch torchvision torchaudio
#Updated cuda and drivers following the github
commands
sudo apt-get install ros-humble-turtlebot3-
description
```

Uso Simulación:

Ejecutar la simulación usando **create3_sim**. Para ejecutar la simulación con el robot create 3, ejecutar los siguientes comandos cambiando **create3_ws** por el workspace donde hayas instalado create3_sim.

```
# Terminal 1
cd *create3_ws*
source install/setup.bash
ros2 launch irobot_create_gazebo_bringup create3_gazebo.launch.py
```

Inicializar el módulo de SLAM:

```
#Terminal 2: Inicializar SLAM
ros2 launch slam_toolbox online_async_launch.py
use_sim_time:=True
params_file:=./src/create3_sim/irobot_create_common/irobot_create_common_bringup/config/mapper_params_online
```

Si quieres controlar el robot con el teclado:

```
#Terminal 2-A: Inicializar control de movimiento del robot
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Si quieres controlar el robot usando nav2

```
#Terminal 2-B: Lanzar Nav2
ros2 launch nav2_bringup navigation_launch.py use_sim_time:=True
```

Nota: Para controlar con NAV2 se usa rviz, ya está configurado para trabajar con este paquete. Los obstáculos para la generación del mapa añadirlos desde Gazebo.

Ficheros importantes en **create3_sim**:

- */create3_sim/irobot_create_common/irobot_create_control/config/control.yalm*. Parámetros para el controlador del robot.
- */create3_sim/irobot_create_gazebo/irobot_create_gazebo_bringup/launch/worlds*: Modelos de entornos para gazebo.
- *create3_sim/irobot_create_common/irobot_create_description/urdf/create3.urdf.xacro*: Modelo robot.

- `create3_sim/irobot_create_common/irobot_create_common_bringup/config/mapper_params_online`: Archivo de configuración para el paquete de SLAM.

Nota: Ciertos ficheros están cambiados con respecto a los archivos generales. Como es github se puede ver los cambios fácilmente.

Uso RL:

Este paquete se puede usar para entrenar y testear un agente de RL. Para usarlo ejecutar los siguientes comandos en terminales diferentes. Además de `*create3_ws*` cambiar también `*rl_ws*` al workspace donde se haya instalado `turtlebot3_drlnav`

Ejecutar simulación usando uno de los entornos definidos para RL:

Terminal 1

```
. ~/*create3_ws*/src/create3_sim/setup.sh
cd *create3_ws*
source install/setup.bash
source ~/*rl_ws*/turtlebot3_drlnav/setup_drlnav.sh
# This has to contain the same number as the stage used
touch /tmp/drlnav_current_stage.txt
echo 9 > /tmp/drlnav_current_stage.txt
# Opción con obstáculos
ros2 launch irobot_create_gazebo_bringup create3_gazebo.launch.py
world_path:=/home/raul/create3_ws/src/create3_sim/irobot_create_
gazebo/irobot_create_gazebo_bringup/launch/worlds/stage9-
obstacle.model
```

Ejecutar el entorno de RL:

#Terminal 2

```
. ~/*create3_ws*/src/create3_sim/setup.sh
conda activate turtle3-drlnav
cd ~/*rl_ws*/turtlebot3_drlnav
source install/setup.bash
source setup_drlnav.sh
ros2 run turtlebot3_dr1 environment
```

Inicializar el entrenamiento:

#Terminal 3

```
. ~/*create3_ws*/src/create3_sim/setup.sh
conda activate turtle3-drlnav
```

```

cd ~/*rl-ws*/turtlebot3_drlnav
source install/setup.bash
source setup_drlnav.sh
ros2 run turtlebot3_drl train_agent ddpq
# En caso de seguir entrenando desde un punto
# ros2 run turtlebot3_drl train_agent ddpq "ddpg_0" 500
#En caso de ejecutar un ejemplo
#ros2 run turtlebot3_drl test_agent ddpq
'examples/ddpg_0_stage9' 8000
# En caso de ejecutar un agente ya entrenado
# ros2 run turtlebot3_drl test_agent ddpq "ddpg_5_stage_9"
15300

```

Ejecutar el nodo gazebo_goals que se encargará de mandar los puntos a los que se tendrá que ir moviendo el robot durante el entrenamiento.

```

#Terminal 4
. ~/*create3_ws*/src/create3_sim/setup.sh
conda activate turtle3-drlnav
cd ~/*rl-ws*/turtlebot3_drlnav
source install/setup.bash
source setup_drlnav.sh
ros2 run turtlebot3_drl gazebo_goals

```

Ficheros importantes **RL**, todos están dentro de turtlebot_drl y distribuidos por diferentes carpetas:

- **[settings.py](#)**: Fichero de configuración para los parámetros del entrenamiento (learning rate, batch_size, epsilon...), para el entorno de RL y para la simulación.
- **[reward.py](#)**: Aquí se define la función recompensa. Super importante.
- **[drl_agent.py](#)**: No me metí mucho con esta, pero básicamente define el agente que se encarga de definir las acciones que se deben ejecutar en cada paso.
- **[drl_enviroment.py](#)**: Una única clase que se encarga de la gestión del entorno. Recibe y se suscribe a varios topics cuya información luego utiliza para definir y gestionar el entorno. Consta de las siguientes partes:
 - **Inicialización variables**: Define las variables iniciales y aquellas que tendrán asignado un valor asignado a uno de los topics.

- **Inicialización topics**: Define topics publisher/subs/clients.. y los asigna a variables del entorno.
- **Define varias funciones de callback**: Estas funciones se van llamando cuando se cumple su condición de callback. Entiendo que la condición de callback de estas funciones es que se recibe un mensaje, es decir que al final se encargan de gestionar que hacer con los mensajes de los topics (goal pose, goal comm, obstacle odom, odom, scan, callback...)
- **get_state**: Función muy importante, define el estado de nuestra aplicación. Como lo hace es simplemente usando append sobre el array state. Define las condiciones de éxito/fracaso para definir las dentro del estado.
- **initilize_episode**: Pone las cosas que tienen que estar a 0 a 0 al principio del episodio.
- **step_comm_callback**: Es un callback que se llama cada vez que se produce un step entiendo y se recibe un mensaje por el siguiente topic (**step_comm_callback**). Este topic me imagino que es el encargado de llamar a los pasos de la simulación. Es el encargado de simplemente gestionar un step: Obtener nuevo estado, obtener la recompensa.
- **ddpg.py** o sus equivalentes **td3.py** y **dqn.py**.
 - **Actor**: Define la red neuronal que se hará el papel de actor (actor define policy)
 - **Critic**: Define la red neuronal que hará el papel de critic (define la value function si no recuerdo mal)
 - **DDPG**: Define arquitectura DDPG, actor-critic, targets y optimizador, además de los siguientes métodos:
 - **get_action**: Usando el estado definido en state se llama a actor para definir la action. Además si se está entrenando se mete un ruido para ayudar a la red a entrenar.
 - **get_action_random**: Devuelve una acción random.
 - **train**: Se encarga del entrenamiento:
 - Realiza los pasos típicos de DDPG/Q-learning: Obtiene la siguiente acción usando la actor. Obtiene el siguiente valor de Q usando la critic. Actualiza el valor usando la bellman equation.
 - Realiza un paso de entrenamiento de la actor y la critic.
- **drl_gazebo.py**: Es el nodo que se encarga de gestionar gazebo y comunicar gazebo con nuestro entorno DRL muy importante también.